

#01 CPP Basics Part1

김성수

4thrunyul@gmail.com

0. 목차



1. C++ 기초 문법

- 입출력
- 자료형
- 연산자
- 조건문
- 반복문(while문, for문)

1. C++ 기초 문법: Hello World!

solving > debug.cpp > ...

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      cout << "Hello, World!";
7
8      return 0;
9  }
10
11 // 프로그래밍의 가장 기초가 되는 Hello, World!를 출력하는 프로그램
12
13 // 단일 행에 주석을 쓰고 싶다면 앞에 //를 붙여주면 되고,
14
15 /*
16  여러 행에 주석을 쓰고 싶다면
17  이렇게 쓰시면 됩니다!
18  */
```

표준 라이브러리

(STL, Standard Library)

- 프로그램에 필요한 '도구 상자'를 가져오는 코드
- #include <iostream>가 없다면 밑의 cout 함수를 사용할 수 없음

main 함수

- 프로그램을 실행할 때 가장 먼저 찾아가서 시작하는 지점
- 모든 C++ 프로그램에는 반드시 하나의 main 함수가 존재해야 함

종료 신호

- main() 함수에서 프로그램이 운영체제에게 결과를 돌려주는 문장
- 숫자 0은 '프로그램에 문제 없음'을 뜻하는 약속된 신호

Input:

Copy

Expected Output:

Copy

Hello, World!

1. C++ 기초 문법: Hello World!

```
solving > debug.cpp > ...
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      cout << "Hello, World!";
7
8      return 0;
9  }
10
11 // 프로그래밍의 가장 기초가 되는 Hello, World!를 출력하는 프로그램
12
13 // 단일 행에 주석을 쓰고 싶다면 앞에 //를 붙여주면 되고,
14
15 /*
16  여러 행에 주석을 쓰고 싶다면
17  이렇게 쓰시면 됩니다!
18  */
```

네임스페이스

- std : STL에 속하는 모든 함수 앞에 붙여줘야 하는 식별자
- 쓰기 귀찮으므로 앞에서 선언
- 성명에서 '성'에 해당한다고 생각하면 쉬움

주석

- 컴퓨터는 읽지 못하는 글
- //(단일 행 주석) : 한 줄의 짧은 설명을 적을 때 사용
- /* */(다중 행 주석) : 여러 줄에 걸친 긴 설명을 적을 때 사용

1. C++ 기초 문법: Hello World!



```
solving > debug.cpp > main()
1  #include <iostream>
2
3  int main() {
4      std::cout << "Hello, World!";
5
6      return 0;
7  }
```

Input: [Copy](#)

Expected Output: [Copy](#)

Hello, World!

만약 using namespace std;가 없다면...

1. C++ 기초 문법: 입출력

C++ 입출력

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7
8      cin >> n;
9      cout << n;
10
11     return 0;
12 }
```

C 입출력

```
solving > debug.cpp > main()
1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6      printf("%d", n);
7
8      return 0;
9  }
```

입력

- cin >> 변수명;

출력

- cout << 변수명;

C++, C 입출력의 차이점

- C++은 자료형을 스스로 추론하여 알아서 출력
- C는 서식 지정자를 직접 명시해야 함.
- cstdio는 STL에 속해 있지 않기 때문에 namespace가 필요 없음

Input:

12

Copy

Expected Output:

12

Copy

1. C++ 기초 문법: 입출력



```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int a, b, c;
7
8      // 여러 개의 값을 한 번에 입력받기
9      cin >> a >> b >> c;
10
11     // 여러 개의 값을 한 번에 출력하기
12     cout << a << " " << b << " " << c;
13
14     return 0;
15 }
```

Input:

1 2 3

Copy

Expected Output:

1 2 3

Copy

다중 입출력

- >>과 <<을 연속해서 사용하여 다중 입출력을 구현할 수 있음

1. C++ 기초 문법: 입출력

```
solving > debug.cpp > main()
```

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int a, b, c;
7
8      // 여러 개의 값을 한 번에 입력받기
9      cin >> a >> b >> c;
10
11     // 여러 개의 값을 한 번에 출력하기
12     cout << a << "\n" << b << "\n" << c;
13
14     return 0;
15 }
```

“\n”
- 줄바꿈 출력

Input:

1 2 3

Copy

Expected Output:

1
2
3

Copy

1. C++ 기초 문법: 입출력



실습

- 2557 Hello World
- 1000 A+B
- 10998 A*B

1. C++ 기초 문법: 정수형



```
solving > G+ debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 1. 정수형
7      int a;
8
9      long long b;
10
11     return 0;
12 }
```

정수형

- 소수점이 없는 참값(정확한 값) 저장

int

- 범위: -2,147,483,648 ~ 2,147,483,647
- 근사치: 약 21억(2×10^9)

long long

- 범위: -9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
- 근사치: 약 900경(9×10^{18})

1. C++ 기초 문법: 실수형



```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 2. 실수형
7      float c;
8
9      double b;
10
11     return 0;
12 }
```

실수형

- 소수점이 포함될 수도 있는 참값 + 근삿값 저장

float

- 유효 자릿수 : 7자리

double

- 유효 자릿수 : 15자리

유효 자릿수

- 소수점 아래 개수가 아니라 왼쪽에서부터 의미 있는 숫자의 개수
- ex1) float c = 1.23456789;에서 1.234567까지는 참값, 89는 근삿값으로 계산
- ex2) float c = 12345678.9;에서 1234567까지는 참값, 8.9는 근삿값으로 계산
- ex3) float c = 0.00123456789;에서 0.001234567까지는 참값, 89는 근삿값으로 계산

1. C++ 기초 문법: 실수형

```
1  #include <iostream>
2  #include <iomanip>
3
4  using namespace std;
5
6  int main() {
7      double num1 = 246.13579;
8      double num2 = 12.345;
9
10     cout << fixed << setprecision(2) << num1 << "\n";
11
12     cout << setprecision(4) << num1 << "\n";
13
14     cout << defaultfloat << setprecision(4) << num2;
15
16     return 0;
17 }
```

Input:

Copy

Expected Output:

Copy

```
246.14
246.1358
12.35
```

setprecision(n) | <iomanip>

- 출력할 숫자의 자릿수를 결정하는 조정자
- 표시할 마지막 자릿수의 다음 숫자를 확인하여 반올림을 수행
- fixed와 함께 사용될 때 소수점 아래의 출력 범위를 확정

fixed | <iomanip>

- 소수점의 위치를 고정함.
- setprecision의 기준을 '전체 유효 숫자'에서 '소수점 아래 자리수'로 변경

defaultfloat | <iomanip>

- fixed 상태를 해제하고 기본값으로 변경하는 조정자
- setprecision은 초기화 하지 않음

1. C++ 기초 문법: 논리형

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 3. 논리형
7      bool e = true;
8
9      bool f = false;
10
11     return 0;
12 }
```

논리형

- 참(true)과 거짓(false) 두 가지 상태만 존재
- 프로그램의 실행 흐름과 조건 판단을 결정하는 최소 단위

bool

- 0은 false, 0이 아닌 모든 숫자는 true로 변환
- ex1) bool f = 0; //false
- ex2) bool e = 1; //true
- ex3) bool e = -2; //true
- ex4) bool e = 3.14; //true

1. C++ 기초 문법: 문자형

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 4. 문자형
7      char g = 'A';
8
9      char h = '?';
10
11     return 0;
12 }
```

문자형

- 단 하나의 문자를 저장
- 내부적으로는 문자에 해당되는 ASCII 코드의 정수값을 보관

char

- 작은 따옴표(")를 사용해 선언
- 정수를 통해 선언 가능
- 본질적으로는 정수로 저장되기에 산술 연산 가능
- ex1) char g = "A" 오류 발생
- ex2) char g = 'ABC' 오류가 발생하지는 않지만 쓰레기값 저장
- ex3) char g = 65 char g = 'A'와 같음
- ex4) char g = 'A' + 1 char g = 'B'와 같음

1. C++ 기초 문법: 산술 연산자

```
solving > G+ debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 1. 산술 연산자
7
8      int a = 10;
9      int b = 3;
10
11      cout << a + b << " ";
12      cout << a - b << " ";
13      cout << a * b << " ";
14      cout << a / b << " ";
15      cout << a % b;
16
17      return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

13 7 30 3 1

산술 연산자

- 수치 계산을 수행하는 연산자
- 더하기(+), 빼기(-), 곱하기(*), 나누기(/), 나머지(%) 연산이 있음

1. C++ 기초 문법: 산술 복합 대입 연산자



```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 1. 산술 연산자
7
8      int a = 10;
9
10     a = a + 5; // 15
11     a = a - 3; // 12
12     a = a / 4; // 3
13
14     return 0;
15 }
```

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 2. 산술 복합 대입 연산자
7
8      int a = 10;
9
10     a += 5; // 15
11     a -= 3; // 12
12     a /= 4; // 3
13
14     return 0;
15 }
```

산술 복합 대입 연산자

- 산술 연산과 대입을 동시에 수행하여 코드를 간결하게 만드는 연산자

1. C++ 기초 문법: 비교 연산자

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 3. 비교 연산자
7
8      int a = 10;
9      int b = 20;
10     bool check;
11
12     check = (a == b);    // false
13     check = (a != b);    // true
14     check = (a < b);     // true
15     check = (a >= b);    // false
16
17     return 0;
18 }
```

비교 연산자

- 두 값의 크기를 비교하여 결과값으로 참 또는 거짓을 반환하는 연산자

==

- 두 값이 같으면 true, 다르면 false 반환

!=

- 두 값이 다르면 true, 같으면 false 반환

>, <

- 초과, 미만

>=, <=

- 이상, 이하

1. C++ 기초 문법: 논리 연산자

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 4. 논리 연산자
7
8      bool check1 = true;
9      bool check2 = false;
10     bool result;
11
12     result = check1 && check2; // false
13     result = check1 || check2; // true
14     result = !check1;         // false
15
16     return 0;
17 }
```

논리 연산자

- 여러 조건들의 참·거짓 상태를 결합하거나 반전시켜 최종적으로 하나의 결론을 도출하는 연산자

&& (and)

- 둘 다 true일 때 true 반환

|| (or)

- 둘 중 하나라도 true면 true 반환

! (not)

- true는 false로, false는 true로 변환

1. C++ 기초 문법: 논리 연산자

```
solving > debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 4. 논리 연산자
7
8      int score = 75;
9      bool result1, result2, result3;
10
11      result1 = (score > 70) && (score <= 80); // true
12      result2 = (score % 5 == 0) || (score > 90); // true
13      result3 = !(score > 70); // false
14
15      return 0;
16 }
```

result1

- score가 70점 초과이면서 80점 이하인가?

result2

- score가 5의 배수이거나 90점 이상인가?

result3

- score가 70점 이하인가?

1. C++ 기초 문법: 증감 연산자

```
solving > G+ debug.cpp > main()
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      // 5. 증감 연산자
7
8      int a = 5;
9      int b = 5;
10
11     a++; // 6
12     ++a; // 7
13
14     b--; // 4
15     --b; // 3
16
17     return 0;
18 }
```

증감 연산자

- 변수의 값을 1씩 증가시키거나 감소시키는 연산자

1. C++ 기초 문법: 자료형, 연산자

실습

- 11382 꼬마 정민
- 10869 사칙연산

1. C++ 기초 문법: 조건문

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 10;
7      int m = 20;
8
9      if(n > m) {
10         cout << "n > m";
11     }
12     else{
13         cout << "n < m";
14     }
15
16     return 0;
17 }
```

if(조건식)

- 주어진 조건식의 참(true) 거짓(false) 여부에 따라 실행 여부를 결정하는 명령

else

- 앞서 제시된 모든 조건들이 거짓일 때 최종적으로 수행하는 명령

Input:

Copy

Expected Output:

Copy

n < m

1. C++ 기초 문법: 조건문



```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int score = 85;
7
8      if(score >= 90) cout << "A";
9      else if(score >= 80) cout << "B";
10     else if(score >= 70) cout << "C";
11     else cout << "F";
12
13     return 0;
14 }
```

else if

- 확인해야 할 조건이 여러 개인 경우 사용
- 위에서부터 순서대로 검사하며, 중간에 하나라도 참이 나오면 해당 블록을 수행하고 if문 탈출

Input:	Copy
Expected Output:	Copy
B	

1. C++ 기초 문법: 조건문



실습

- 1330 두 수 비교하기
- 9498 시험 성적
- 2753 윤년

1. C++ 기초 문법: 반복문

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 5;
7
8      while(n > 0) {
9          cout << n << " ";
10
11         n--;
12     }
13
14     while(true) {
15         cout << "\n" << n;
16
17         break;
18     }
19
20     return 0;
21 }
```

while(조건문)

- 조건문이 참(true)일 동안 계속 수행 됨
- 조건이 간단할 때 유리
- 초기식과 증감식이 외부에 분산 됨

break;

- 루프의 조건식과 상관 없이 가장 가까운 반복문을 탈출함

Input: [Copy](#)

Expected Output: [Copy](#)

```
5 4 3 2 1
0
```

1. C++ 기초 문법: 반복문

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 5;
7
8      for(int i = 0; i < n; i++) {
9          cout << i << " ";
10     }
11
12     return 0;
13 }
```

for(초기식; 조건식; 증감식)

- 조건식이 참(true)일 동안 계속 수행 됨
- 정해진 횟수 만큼 반복하는 로직을 작성할 때 사용
- 초기식, 조건식, 증감식이 포함 됨

Input:

Copy

Expected Output:

Copy

0 1 2 3 4

1. C++ 기초 문법: 반복문

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n = 3;
7      int m = 5;
8
9      for(int i = 0; i < n; i++) {
10         for(int j = 0; j < m; j++) {
11             cout << i + j << " ";
12         }
13
14         cout << "\n";
15     }
16
17     return 0;
18 }
```

중첩 반복문

- 반복문 안에 반복문이 있는 구조
- while문과 for문만 사용할 수도 있고, 섞어서 사용할 수도 있음

Input: [Copy](#)

Expected Output: [Copy](#)

```
0 1 2 3 4
1 2 3 4 5
2 3 4 5 6
```

1. C++ 기초 문법: 반복문



실습

- 2739 구구단
- 8393 합
- 2438 별찍기 - 1
- 2439 별찍기 - 2

1. C++ 기초 문법: 응용

- 10950 A+B - 3

문제

두 정수 A와 B를 입력받은 다음, A+B를 출력하는 프로그램을 작성하시오.

입력

첫째 줄에 테스트 케이스의 개수 T가 주어진다.

각 테스트 케이스는 한 줄로 이루어져 있으며, 각 줄에 A와 B가 주어진다. ($0 < A, B < 10$)

출력

각 테스트 케이스마다 A+B를 출력한다.

예제 입력 1 복사

```
5
1 1
2 3
3 4
9 8
5 2
```

예제 출력 1 복사

```
2
5
7
17
7
```

테스트 케이스의 수가 주어지는 경우

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, num1, num2;
7
8      cin >> n;
9
10     for(int i = 0; i < n; i++) {
11         cin >> num1 >> num2;
12
13         cout << num1 + num2 << "\n";
14     }
15
16     return 0;
17 }
```

1. C++ 기초 문법: 응용

- 10951 A+B - 4

문제

두 정수 A와 B를 입력받은 다음, A+B를 출력하는 프로그램을 작성하시오.

입력

입력은 여러 개의 테스트 케이스로 이루어져 있다.

각 테스트 케이스는 한 줄로 이루어져 있으며, 각 줄에 A와 B가 주어진다. ($0 < A, B < 10$)

출력

각 테스트 케이스마다 A+B를 출력한다.

예제 입력 1 복사

```
1 1
2 3
3 4
9 8
5 2
```

예제 출력 1 복사

```
2
5
7
17
7
```

테스트 케이스의 수가 주어지지 않는 경우
(End of File)

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, num1, num2;
7
8      while(cin >> num1 >> num2) {
9          cout << num1 + num2 << "\n";
10     }
11
12     return 0;
13 }
```

1. C++ 기초 문법: 응용

- 10952 A+B - 5

문제

두 정수 A와 B를 입력받은 다음, A+B를 출력하는 프로그램을 작성하시오.

입력

입력은 여러 개의 테스트 케이스로 이루어져 있다.

각 테스트 케이스는 한 줄로 이루어져 있으며, 각 줄에 A와 B가 주어진다. ($0 < A, B < 10$)

입력의 마지막에는 0 두 개가 들어온다.

출력

각 테스트 케이스마다 A+B를 출력한다.

예제 입력 1 복사

```
1 1
2 3
3 4
9 8
5 2
0 0
```

예제 출력 1 복사

```
2
5
7
17
7
```

마지막에 특정 값이 들어오는 경우

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, num1, num2;
7
8      while(cin >> num1 >> num2) {
9          if(num1 == 0 && num2 == 0) break;
10
11         cout << num1 + num2 << "\n";
12     }
13
14     return 0;
15 }
```

1. C++ 기초 문법: 응용



실습

- 2884 알람시계
- 2480 주사위 3개
- 5073 삼각형과 세 변

1. C++ 기초 문법: 배열



배열(Array)

- 같은 자료형의 변수들을 메모리 상에 연속적으로 나열한 자료구조
- 모든 자료형(int, float, char, bool, string 등)에서 선언 가능
- 데이터가 빈틈없이 붙어 있어 인덱스(번호)만 알면 즉시 찾을 수 있음.
- 고정 크기 : 한 번 크기를 정하면 칸의 개수를 늘리거나 줄일 수 없음.

선언과 초기화

- score1 : int형 배열 3칸을 선언하였지만, 초기화가 되어있지 않음. (= 쓰레기값이 들어 있음)
- score2 : int형 배열 3칸을 선언하였고, 모든 칸에 각 원소를 선언하였음.
- score3 : int형 배열을 선언하되, 초기화 되는 원소의 개수에 따라 칸의 개수가 정해짐.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int score1[3];
7
8      int score2[3] = {10, 20, 30};
9
10     int score3[] = {100, 200, 300, 400, 500};
11
12     return 0;
13 }
```

1. C++ 기초 문법: 배열



```
1  #include <iostream>
2
3  using namespace std;
4
5  int score_Global[5];
6
7  int main() {
8      for(int i = 0; i < 5; i++) {
9          cout << score_Global[i] << " ";
10     }
11
12     return 0;
13 }
```

전역 배열 선언

- 지역 배열 선언과 다르게, 초기화를 따로 해주지 않아도 각 원소들이 0으로 초기화 됨
- 쓰레기값이 저장되어있지 않기 때문에 런타임 에러(Segmentation Fault) 발생을 예방함.

Input:

Copy

Expected Output:

Copy

0 0 0 0 0

1. C++ 기초 문법: 배열

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int size[3] = {10, 20, 30};
7
8      cout << size[0] << " ";
9
10     size[1] = 15;
11     cout << size[1] << " ";
12
13     size[2]++;
14     size[2] += size[0];
15     cout << size[2];
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

10 15 41

배열 인덱싱

- 배열의 첫 번째 요소는 항상 0번에서 시작함.
- 인덱싱을 통해 원소에 접근할 수 있으며, 수정하거나 출력할 수 있음.
- 반복문의 제어 변수(주로 i)를 통해 배열의 모든 원소를 탐색할 수 있음

```
1  #include <iostream>
2  #include <iterator>
3
4  using namespace std;
5
6  int main() {
7      int record[5] = {10, 20, 30, 40, 50};
8
9      for(int i = 0; i < size(record); i++) {
10         cout << record[i] << " ";
11     }
12
13     return 0;
14 }
```

Input:

Copy

Expected Output:

Copy

10 20 30 40 50

1. C++ 기초 문법: 배열



```
1  #include <iostream>
2  #include <iterator>
3
4  using namespace std;
5
6  int main() {
7      int record[5] = {10, 20, 30, 40, 50};
8
9      int check = size(record);
10
11     cout << check;
12
13     return 0;
14 }
```

size(변수명) <iterator>

- 배열 크기 반환

begin(변수명), end(변수명) <iterator>

- 각각 배열의 시작 포인터(주소)와 마지막 칸 다음 칸 포인터를 반환
- 단독으로 쓰이기 보다는 <algorithm>의 fill, reverse 등의 함수와 함께 쓰임

Input:

Copy

Expected Output:

Copy

5

1. C++ 기초 문법: 배열

```
1  #include <iostream>
2  #include <iterator>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      int arr[] = {1, 1, 2, 2, 5, 10};
9
10     int cnt2 = count(begin(arr), end(arr), 2);
11
12     int maxVal = *max_element(begin(arr), end(arr));
13     int minVal = *min_element(begin(arr), end(arr));
14
15     cout << cnt2 << " " << maxVal << " " << minVal;
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

2 10 1

count(시작 주소, 끝 주소, 찾는 값) <algorithm>

- 찾는 값이 해당 배열에 몇 개 있는지 반환

max_element(시작 주소, 끝 주소) <algorithm>

min_element(시작 주소, 끝 주소) <algorithm>

- 각각 해당 배열의 최댓값과 최솟값을 찾고 해당 원소의 포인터를 반환
- 결과값으로 포인터를 반환하기에 앞에 *를 붙여 해당 값에 접근할 수 있음.

1. C++ 기초 문법: 배열



```
1  #include <iostream>
2  #include <iterator>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      int arr[] = {1, 1, 2, 2, 5, 10};
9
10     reverse(begin(arr), end(arr));
11
12     for(int i = 0; i < size(arr); i++) {
13         cout << arr[i] << " ";
14     }
15
16     return 0;
17 }
```

reverse(시작 주소, 끝 주소) <algorithm>
- 배열 뒤집기

Input:

Copy

Expected Output:

Copy

10 5 2 2 1 1

1. C++ 기초 문법: 배열



fill(시작 주소, 끝 주소, 바꿀 값) <algorithm>

- 배열 전체를 특정 값으로 초기화 함
- 주로 전역변수로 배열을 선언할 때 초기값이 0으로 선언되어 있는 것을 변경하고 싶을 때 사용
- 해당 예제에서는 배열의 모든 원소가 -1로 변경 됨.

```
1  #include <iostream>
2  #include <iterator>
3  #include <algorithm>
4
5  int arr[10];
6
7  using namespace std;
8
9  int main() {
10     fill(begin(arr), end(arr), -1);
11
12     return 0;
13 }
```

1. C++ 기초 문법: 배열

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int arr[3][4] = {
7          {1, 2, 3, 4},
8          {5, 6, 7, 8},
9          {9, 10, 11, 12}
10     };
11
12     cout << arr[1][2];
13
14     return 0;
15 }
```

2차원 배열

- 표 형태의 데이터를 다루기 위한 구조
- 배열을 요소로 가지는 배열
- 행과 열로 이루어져 있으며, 1차원 배열과 선언, 초기화, 인덱싱은 동일함
- `arr[i][j]`에서 행과 열이 헷갈리면 건물의 '층 j호'라고 생각하면 쉬움

Input: Copy

Expected Output: Copy

7

1. C++ 기초 문법: 배열



실습

- [10807 개수 세기](#)
- [10818 최소, 최대](#)
- [2738 행렬 덧셈](#)

3. C++ 기초 문법: 문자열

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string s1;
8      string s2 = "ABC";
9      string s3 = "BBC";
10     string s4 = s2 + s3;
11
12     cout << s2 << " " << s3 << " " << s4;
13     cout << "\n" << s2[0] << s4[5];
14
15     return 0;
16 }
```

Input: Copy

Expected Output: Copy

ABC BBC ABCBBC
AC

문자열(String)

- 문자(char)들을 배열처럼 메모리상에 연속적으로 나열한 자료구조
- char[]와 달리, 객체 형태여서 크기 조절이 자유롭고 다양한 편의 기능을 사용할 수 있음.
- 문자열의 길이에 따라 메모리 공간이 자동으로 늘어나거나 줄어듦. 즉, 가변 크기 형태임

선언과 초기화

- string을 사용하기 위해서는 <string> 라이브러리가 필요함.
- char과 다르게 큰 따옴표(“”)를 사용해 선언
- 배열과 다르게 초기화를 하지 않아도 쓰레기값이 들어있지 않음

인덱싱

- 배열처럼 인덱싱을 통해 문자열의 각 문자에 접근할 수 있음

3. C++ 기초 문법: 문자열

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string s1, s2, s3, s4;
8
9      cin >> s1 >> s2 >> s3;
10
11     cin.ignore();
12
13     getline(cin, s4);
14
15     cout << s1 << " " << s2;
16     cout << "\n" << s4;
17
18     return 0;
19 }
```

입력

- cin을 사용하면 다른 입력들과 마찬가지로 공백이나 줄바꿈(Enter) 전까지만 입력 받음
- 띄어쓰기가 포함된 문장을 입력 받고 싶다면 getli

getline(cin, str);

- 공백이 포함된 한 줄 전체를 입력 받고 싶다면 해당 함수를 사용해야 함.
- getline은 '줄바꿈이 나오기 전까지 입력 받아라'라는 명령
- cin을 먼저 앞에 사용했더라면 줄바꿈 버퍼가 남아있기에 명령이 바로 종료됨.
- cin.ignore();을 사용하여 해당 버퍼를 무시할 수 있음

Input:

Copy

won't back down
won't back down

Expected Output:

Copy

won't back
won't back down

3. C++ 기초 문법: 문자열

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string s1 = "Apple";
8      string s2 = "Banana";
9      string s3 = s1;
10
11     bool check1 = (s1 == s3);
12     bool check2 = (s1 > s2);
13
14     cout << check1 << " " << check2;
15
16     return 0;
17 }
```

연산자 활용

- ==을 통해 두 문자열의 동일 여부를 확인할 수 있음.
- >나 <을 통해 두 문자열의 사전 순서를 비교할 수 있음.
- check2에서 s1이 s2 사전 순서가 더 빠르기 때문에 0(false) 출력

Input:

Copy

Expected Output:

Copy

1 0

3. C++ 기초 문법: 문자열

```
1  #include <iostream>
2  #include <string>
3  #include <cctype>
4
5  using namespace std;
6
7  int main() {
8      string s1 = "apple";
9      string s2 = "Do you guys not have phones?";
10
11     cout << s1.size() << " " << s1.length();
12
13     for(int i = 0; i < s2.size(); i++) {
14         if(isupper(s2[i])) {
15             s2[i] = tolower(s2[i]);
16         }
17         else if(islower(s2[i])) {
18             s2[i] = toupper(s2[i]);
19         }
20     }
21
22     cout << "\n" << s2;
23
24     return 0;
25 }
```

str.size(), str.length()

- 문자열의 크기 반환

isupper(char), islower(char) <cctype>

- 해당 문자(char)가 대문자(소문자)인지 판별하는 함수
- 참이면 0이 아닌 양수, 거짓이면 0을 반환

toupper(char), tolower(char) <cctype>

- 해당 문자를 대문자(소문자)로 반환하는 함수

Input: [Copy](#)

Expected Output: [Copy](#)

5 5
dO YOU GUYS NOT HAVE PHONES?

3. C++ 기초 문법: 문자열

```
1  #include <iostream>
2  #include <string>
3
4
5  using namespace std;
6
7  int main() {
8      string s1 = "apple";
9      string s2 = "banana";
10     string s3 = "pineapple";
11
12     int pos1 = s3.find(s1);
13     int pos2 = s3.find(s2);
14
15     if(pos1 != string::npos) cout << s1;
16     if(pos2 != string::npos) cout << s2;
17
18     cout << "\n" << s3.substr(2);
19     cout << "\n" << s3.substr(2, 4);
20
21     return 0;
22 }
```

str.find(찾는 문자열) <string>

- 참이면 해당 문자열이 시작되는 첫 번째 인덱스를 반환
- 거짓이면 string::npos라는 특수한 값 반환

str.substr(n); <string>

- 문자열의 n번째 인덱스부터 출력

str.substr(n, k); <string>

- 문자열의 n번째 인덱스부터 k번 출력

Input: [Copy](#)

Expected Output: [Copy](#)

```
apple
neapple
neap
```

3. C++ 기초 문법: 문자열



실습

- [27866 문자와 문자열](#)
- [11654 아스키 코드](#)
- [2857 FBI](#)
- [11720 숫자의 합](#)

1. C++ 기초 문법: 범위 기반 for문

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string s = "apple";
8
9      for(int i = 0; i < s.size(); i++) {
10         cout << s[i] << " ";
11     }
12
13     cout << "\n";
14
15     for(char c : s) {
16         cout << c << " ";
17     }
18
19     return 0;
20 }
```

for(자료형 변수명 : 컨테이너명)

- 배열이나 문자열의 처음부터 끝까지 모든 원소를 순회할 때 사용하는 방법
- 인덱스를 쓰지 않아서 범위 초과, 미만 오류를 원천 차단할 수 있음
- 배열이나 문자열의 끝에 도달하면 알아서 반복을 멈춤

Input:

Copy

Expected Output:

Copy

a p p l e
a p p l e